Semester Project

Pipelined RISCV Processor (32-bit) with Floating-Point Units

Group Members:

Rabia Samad Afridi

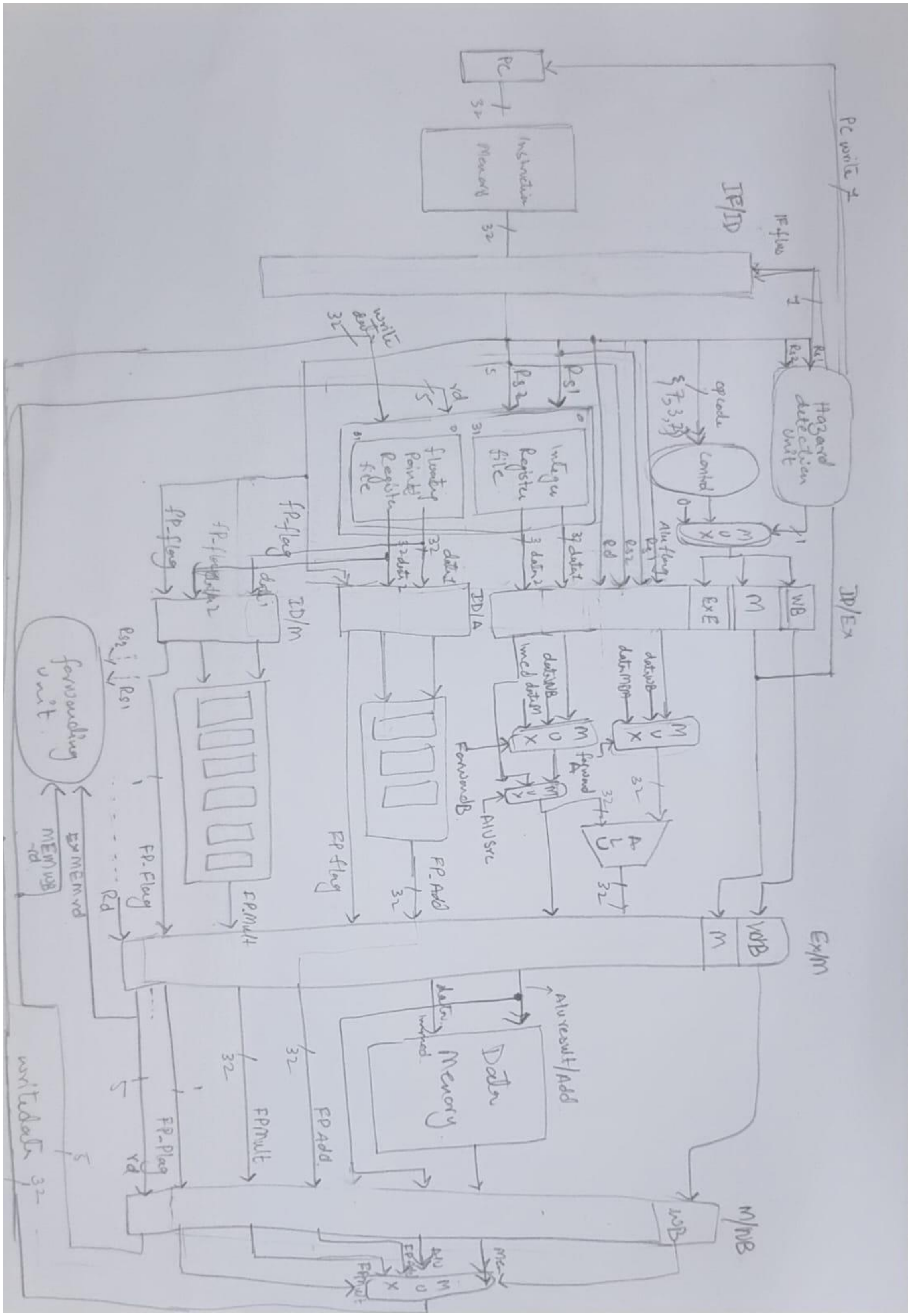
Mehwish Bibi

Maria Khan

Deliverables:

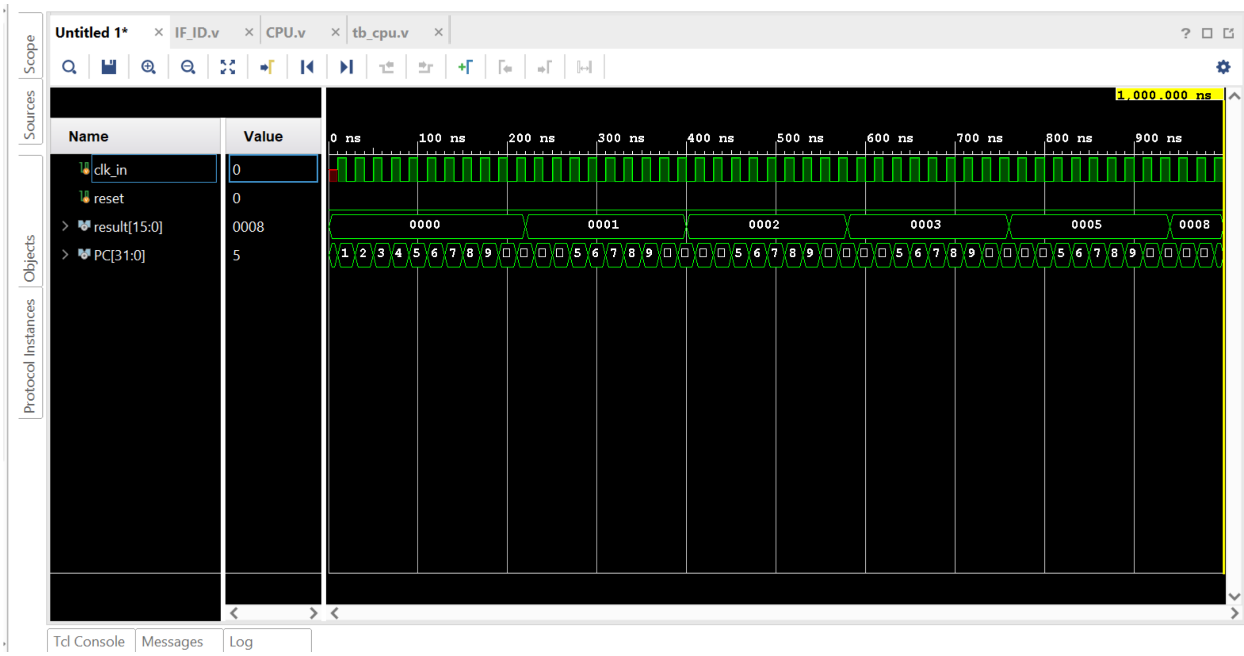
- Pipelining of the RISCV Processor (32-bit) - RV32I base ISA
- Test it using Fibonacci Sequence
- Extend it with addition of floating-point unit (FP_Add/FP_Sub, FP_Mult, FP_Div) floating point units can be taken from any open source
- Implement static scheduling with
  - Data Hazards Detection and Mitigation Support (Forwarding & Stall)
  - Control Hazards Detection and Mitigation Support (Dynamic branch prediction - Single Bit)

Design of RISC V with floating point Units

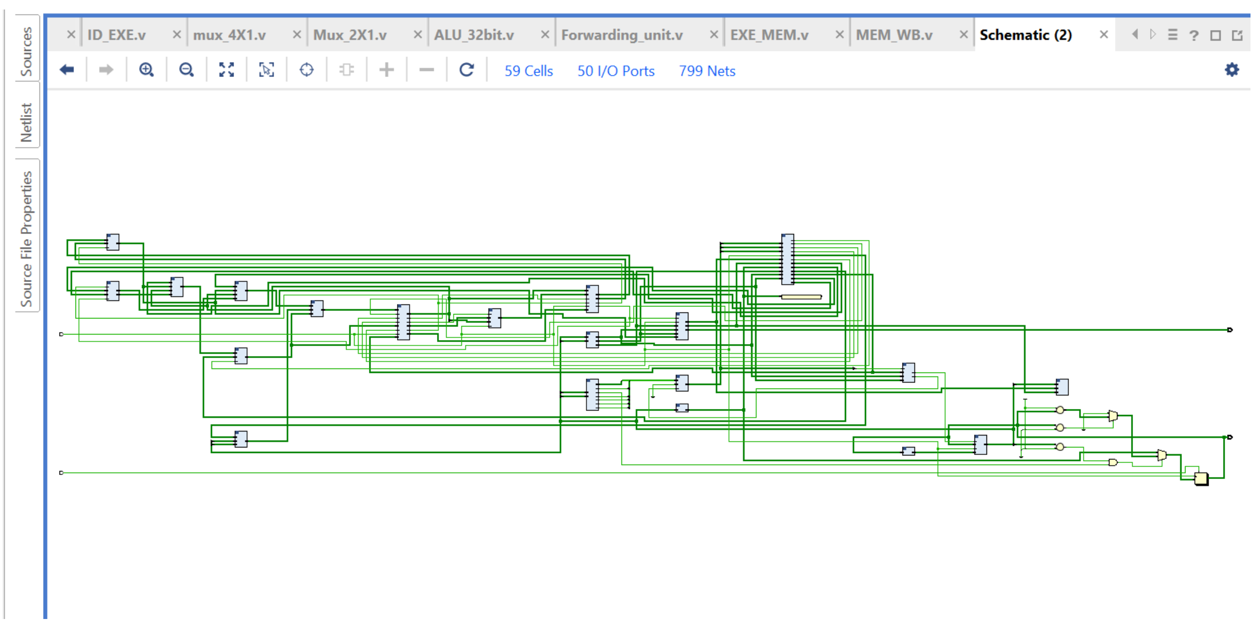


1. Pipelining of the RISCV Processor (32-bit) - RV32I base ISA

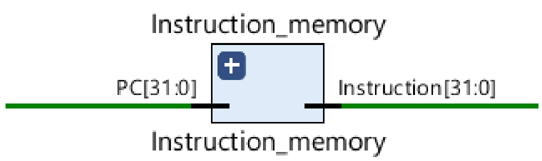
Timing diagram of Fibonacci series using pipelined architecture

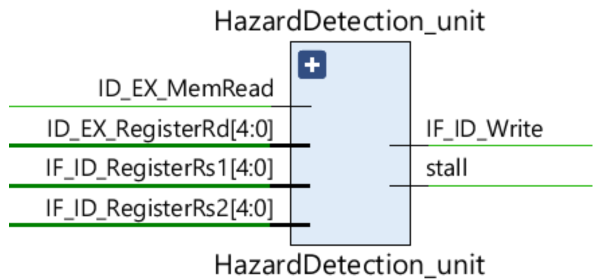


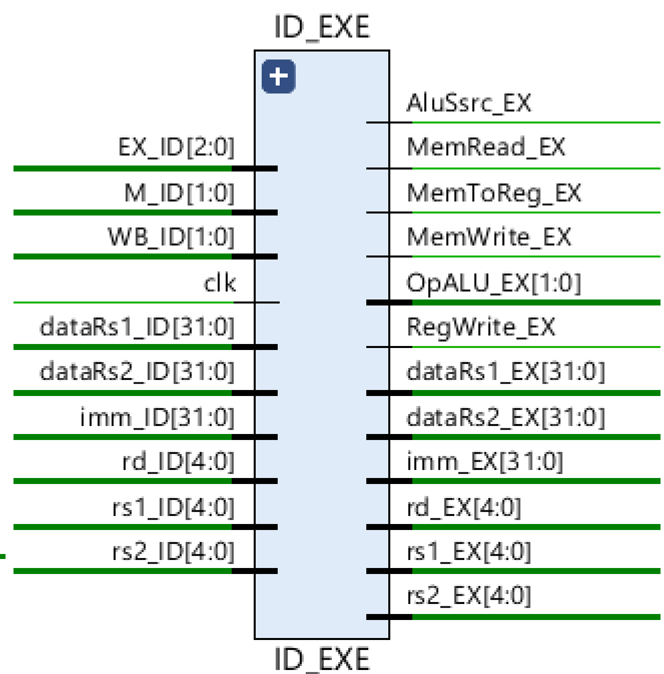
RTL Schematic of pipelined architecture

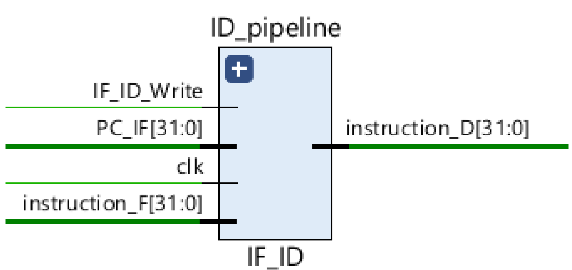


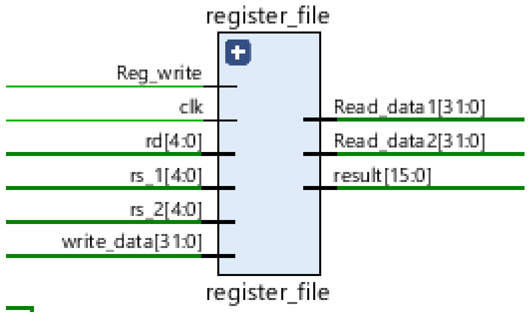
Different units close up pictures

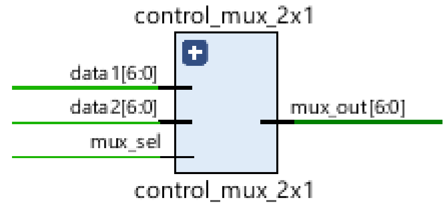


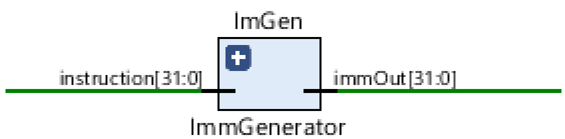


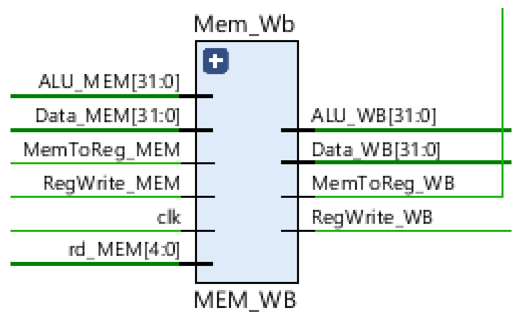


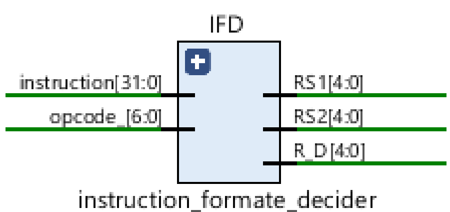


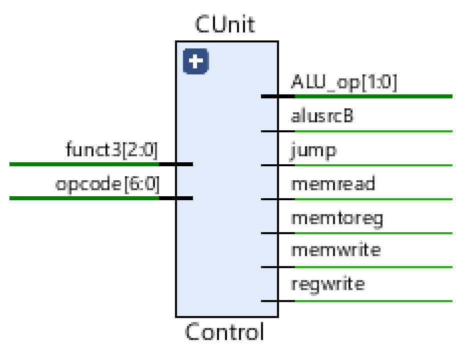

register_file
Reg_write
clk
rd[4:0]
rs_1[4:0]
rs_2[4:0]
write_data[31:0]
Read_data1[31:0]
Read_data2[31:0]
result[15:0]
register_file

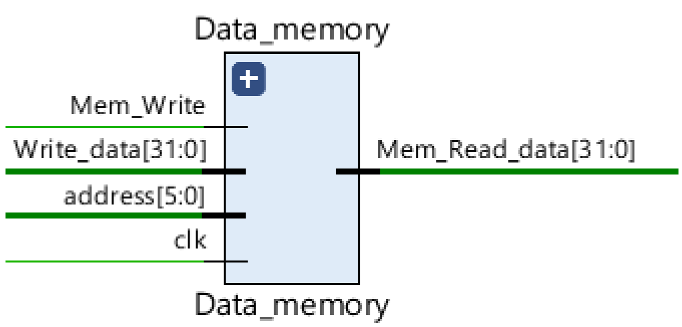

control_mux_2x1
data1[6:0]
data2[6:0]
mux_sel
mux_out[6:0]
control_mux_2x1

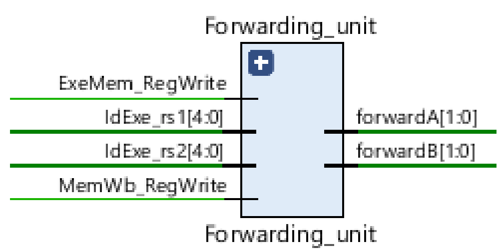

ImGen
instruction[31:0]
immOut[31:0]
ImmGenerator

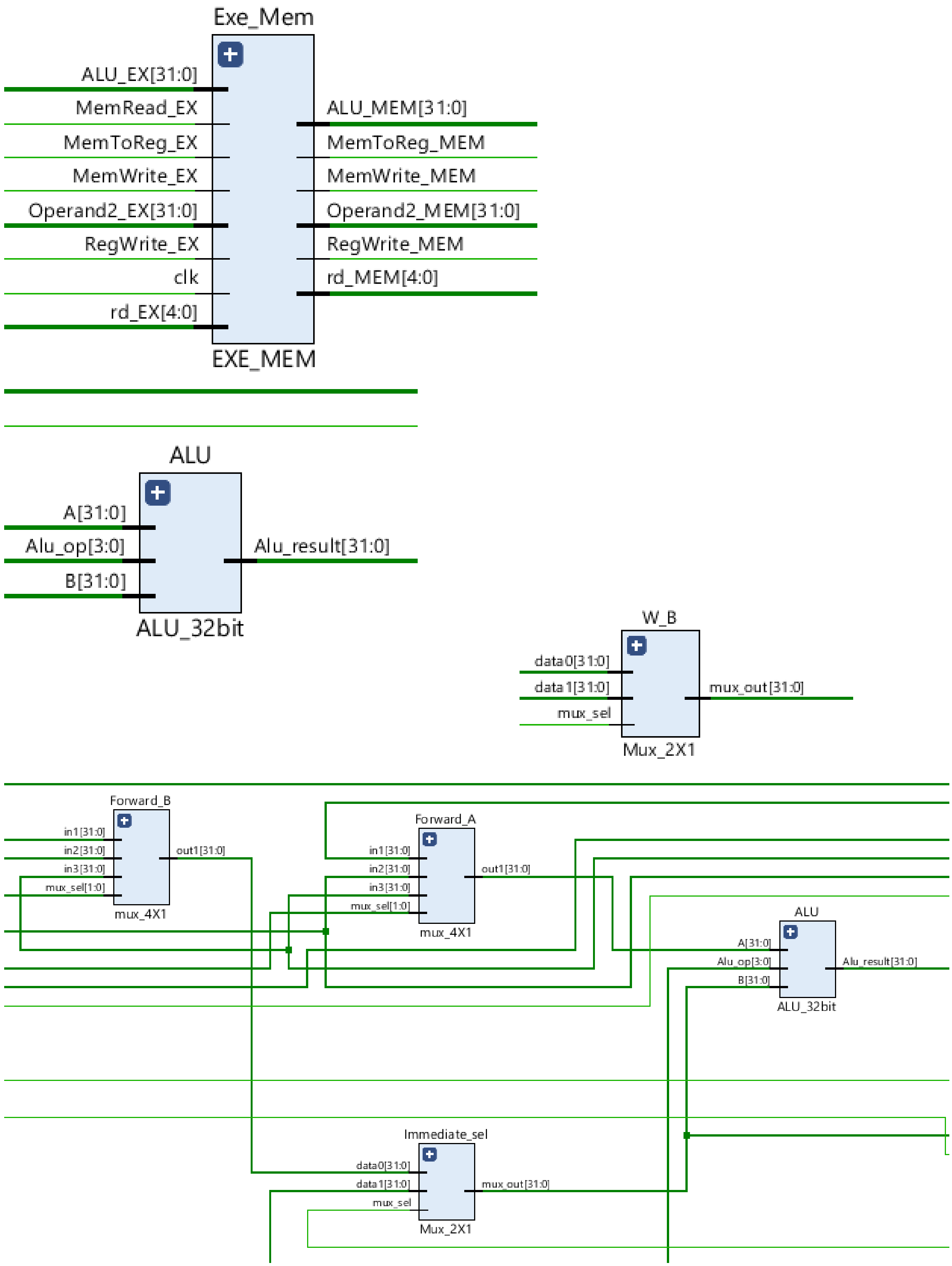

Mem_Wb
ALU_MEM[31:0]
Data_MEM[31:0]
MemToReg_MEM
RegWrite_MEM
clk
rd_MEM[4:0]
ALU_WB[31:0]
Data_WB[31:0]
MemToReg_WB
RegWrite_WB
MEM_WB


IFD
instruction[31:0]
opcode_[6:0]
RS1[4:0]
RS2[4:0]
R_D[4:0]
instruction_formate_decider


CUnit
funct3[2:0]
opcode[6:0]
ALU_op[1:0]
alusrcB
jump
memread
memtoreg
memwrite
regwrite
Control


Data_memory
Mem_Write
Write_data[31:0]
address[5:0]
clk
Mem_Read_data[31:0]
Data_memory


Forwarding_unit
ExeMem_RegWrite
IdExe_rs1[4:0]
IdExe_rs2[4:0]
MemWb_RegWrite
forwardA[1:0]
forwardB[1:0]
Forwarding_unit

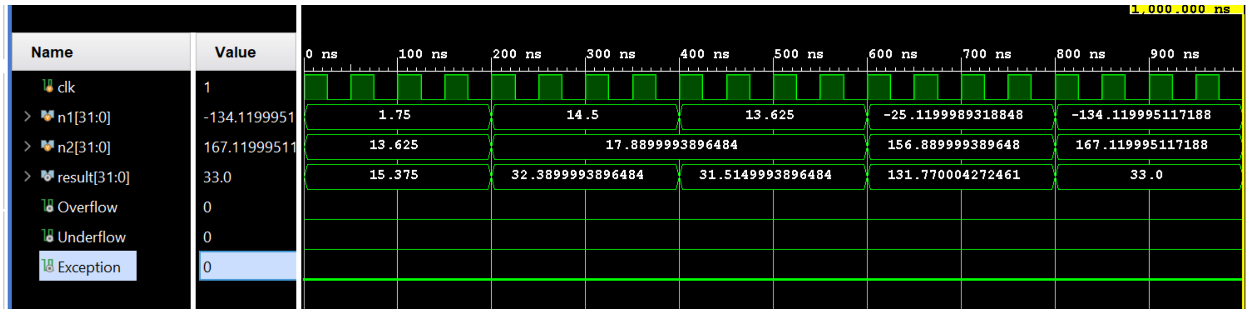


Exe_Mem
ALU_EX[31:0]
MemRead_EX
MemToReg_EX
MemWrite_EX
Operand2_EX[31:0]
RegWrite_EX
clk
rd_EX[4:0]
ALU_MEM[31:0]
MemToReg_MEM
MemWrite_MEM
Operand2_MEM[31:0]
RegWrite_MEM
rd_MEM[4:0]
EXE_MEM
ALU
A[31:0]
Alu_op[3:0]
B[31:0]
Alu_result[31:0]
ALU_32bit
W_B
data0[31:0]
data1[31:0]
mux_sel
mux_out[31:0]
Mux_2X1
Forward_B
in1[31:0]
in2[31:0]
in3[31:0]
mux_sel[1:0]
out1[31:0]
mux_4X1
Forward_A
in1[31:0]
in2[31:0]
in3[31:0]
mux_sel[1:0]
out1[31:0]
mux_4X1
ALU
A[31:0]
Alu_op[3:0]
B[31:0]
Alu_result[31:0]
ALU_32bit
Immediate_sel
data0[31:0]
data1[31:0]
mux_sel
mux_out[31:0]
Mux_2X1

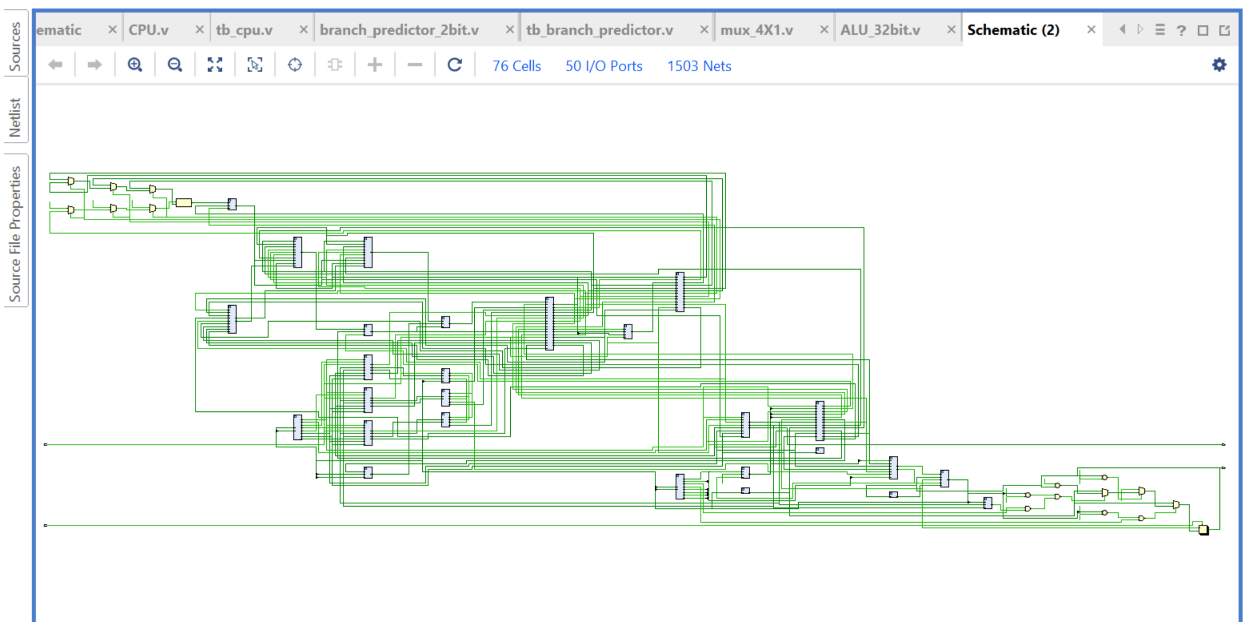
2. **Extend it with addition of floating-point unit (FP_Add/FP_Sub, FP_Mult, FP_Div) floating point units**

3.

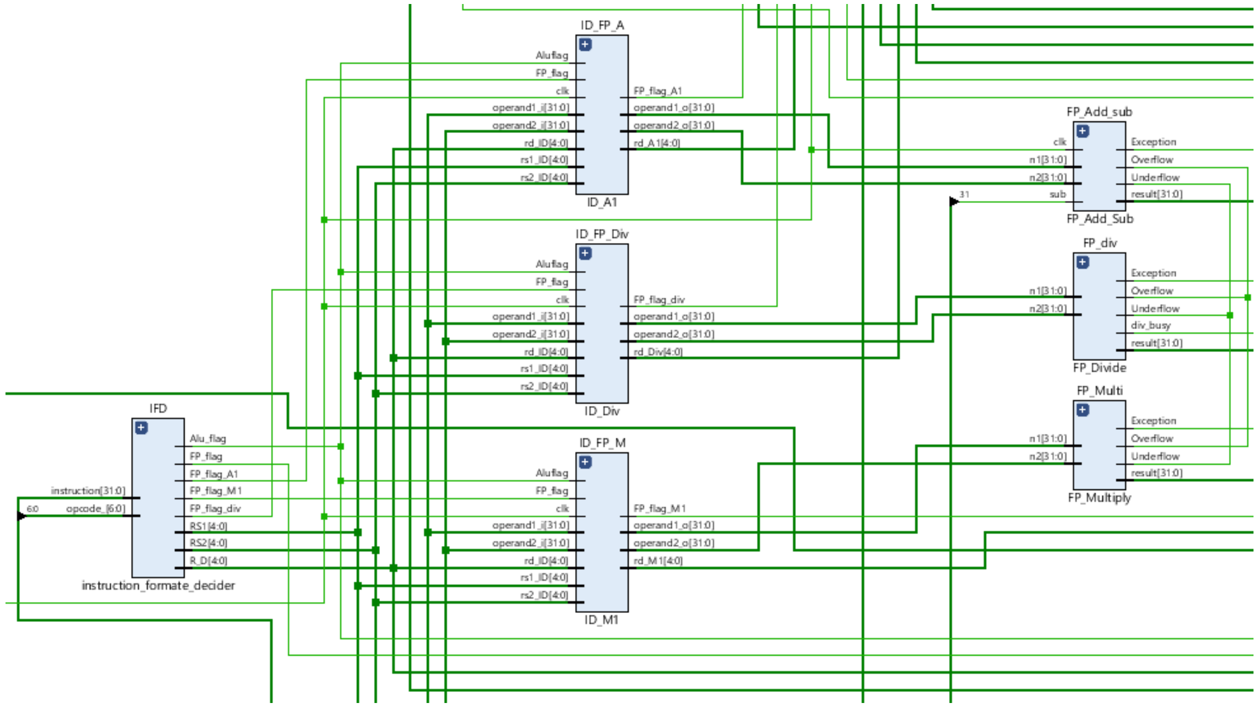
Timing Diagram of vector addition

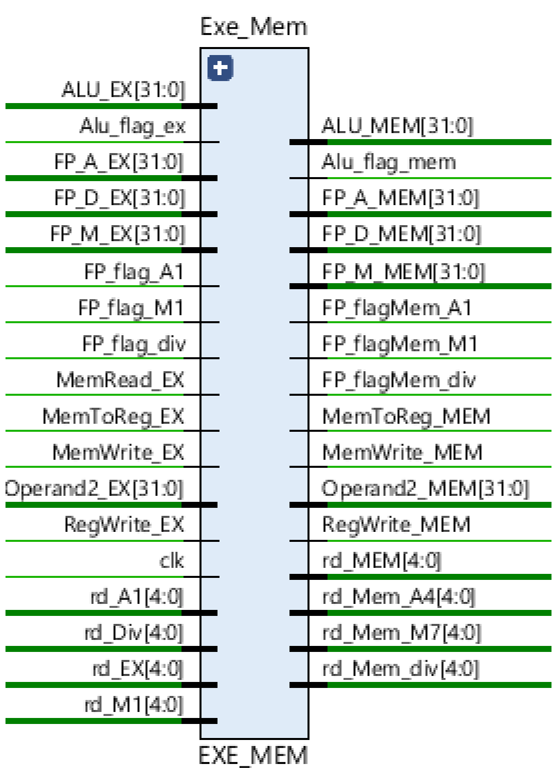


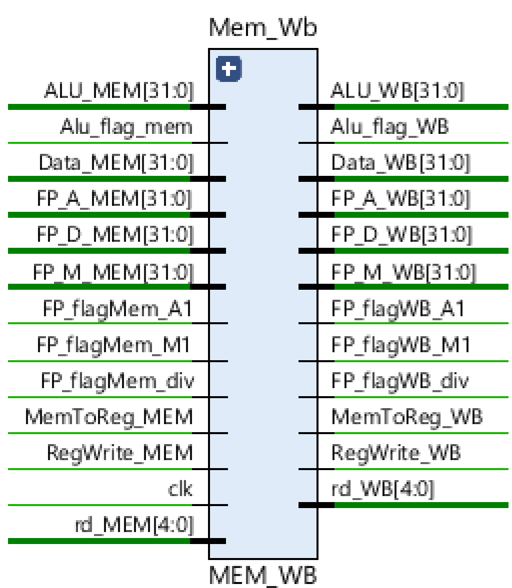
RTL Schematic of pipelined architecture With Floating Point Units

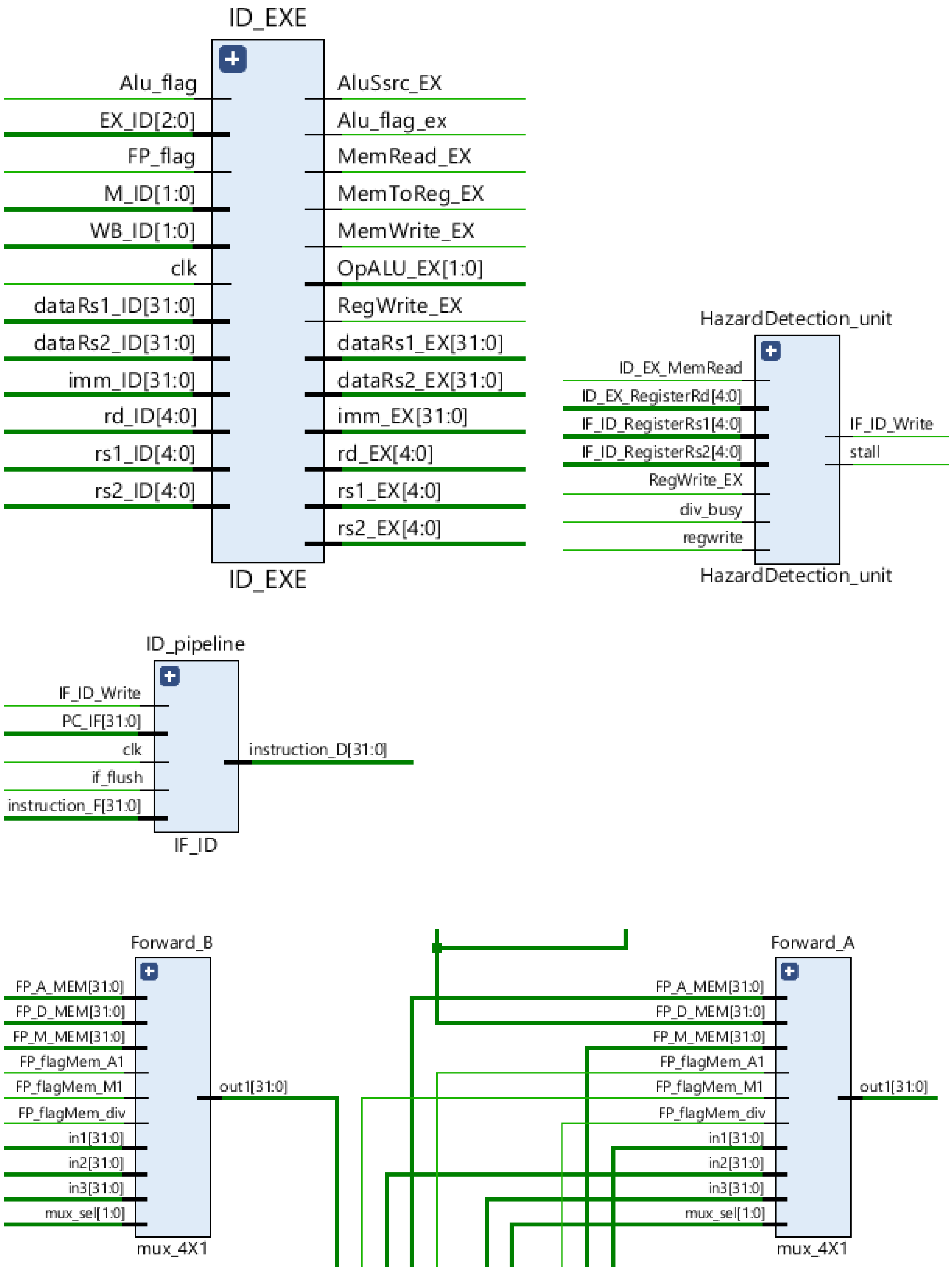


Added Part in Pipelined Architecture








ID_EXE
Alu_flag
EX_ID[2:0]
FP_flag
M_ID[1:0]
WB_ID[1:0]
clk
dataRs1_ID[31:0]
dataRs2_ID[31:0]
imm_ID[31:0]
rd_ID[4:0]
rs1_ID[4:0]
rs2_ID[4:0]
AluSsrc_EX
Alu_flag_ex
MemRead_EX
MemToReg_EX
MemWrite_EX
OpALU_EX[1:0]
RegWrite_EX
dataRs1_EX[31:0]
dataRs2_EX[31:0]
imm_EX[31:0]
rd_EX[4:0]
rs1_EX[4:0]
rs2_EX[4:0]
ID_EXE
HazardDetection_unit
ID_EX_MemRead
ID_EX_RegisterRd[4:0]
IF_ID_RegisterRs1[4:0]
IF_ID_RegisterRs2[4:0]
RegWrite_EX
div_busy
regwrite
IF_ID_Write
stall
HazardDetection_unit
ID_pipeline
IF_ID_Write
PC_IF[31:0]
clk
if_flush
instruction_F[31:0]
instruction_D[31:0]
IF_ID
Forward_B
FP_A_MEM[31:0]
FP_D_MEM[31:0]
FP_M_MEM[31:0]
FP_flagMem_A1
FP_flagMem_M1
FP_flagMem_div
in1[31:0]
in2[31:0]
in3[31:0]
mux_sel[1:0]
out1[31:0]
mux_4X1
Forward_A
FP_A_MEM[31:0]
FP_D_MEM[31:0]
FP_M_MEM[31:0]
FP_flagMem_A1
FP_flagMem_M1
FP_flagMem_div
in1[31:0]
in2[31:0]
in3[31:0]
mux_sel[1:0]
out1[31:0]
mux_4X1